Pierre-Alexandre Monceau

Rapport Projet Traitement Image :

Compilation : make all

Exécution : ./Tp

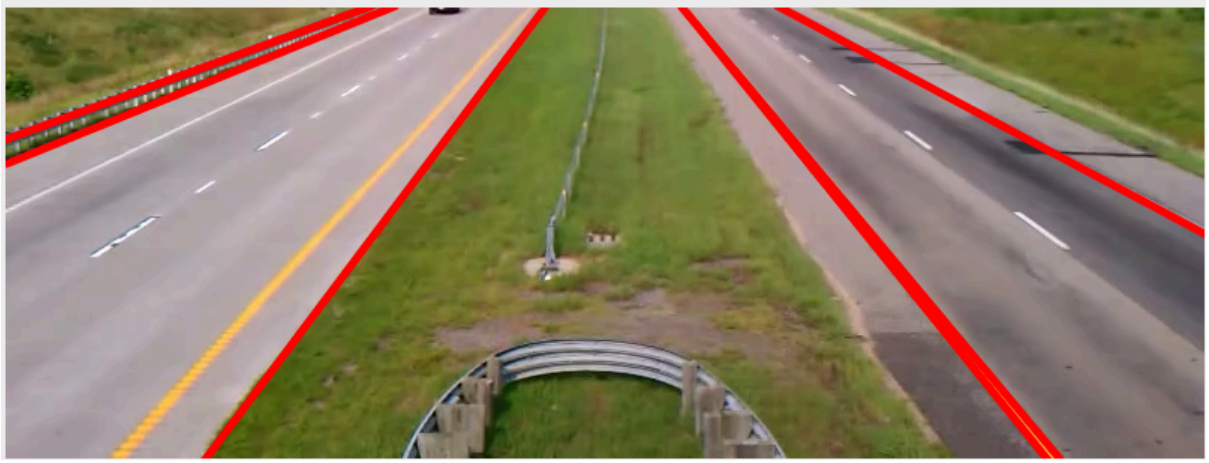
I. Détection des voies :

- J'ai commencé par appliquer un filtre de Canny pour ne garder que les contours.
https://docs.opencv.org/4.x/da/d22/tutorial_py_canny.html

- J'ai récupéré la 10eme image en faisant un compteur.
J'applique ensuite une transformation de Hough pour garder les plus grandes lignes droites qui correspondent aux bordures de la route.
https://docs.opencv.org/3.4/d9/db0/tutorial_hough_lines.html

- On dessine ensuite ces lignes sur chaque frame avec la fonction lines().

Résultat : On obtient bien la détection des deux voies rapides par détection de leurs bordures.
La voie de gauche est plus facile à détecter que celle de droite.
Ainsi, j'obtiens une ligne 'parasite' sur la voie de gauche afin de pouvoir détecter correctement la voie de droite.



II. Suivi des véhicules :

- Suppression du background avec la classe opencv BackgroundSubtractorMOG2
https://docs.opencv.org/4.x/d1/dc5/tutorial_background_subtraction.html

- floutage médian afin d'éliminer le bruit de type poivre et sel avec la fonction cv::medianBlur
https://docs.opencv.org/3.4/d4/d86/group__imgproc__filter.html#ga564869aa33e58769b4469101aac458f9

- Binarisation avec un seuil grâce à la fonction cv::threshold
https://docs.opencv.org/4.x/db/d8e/tutorial_threshold.html

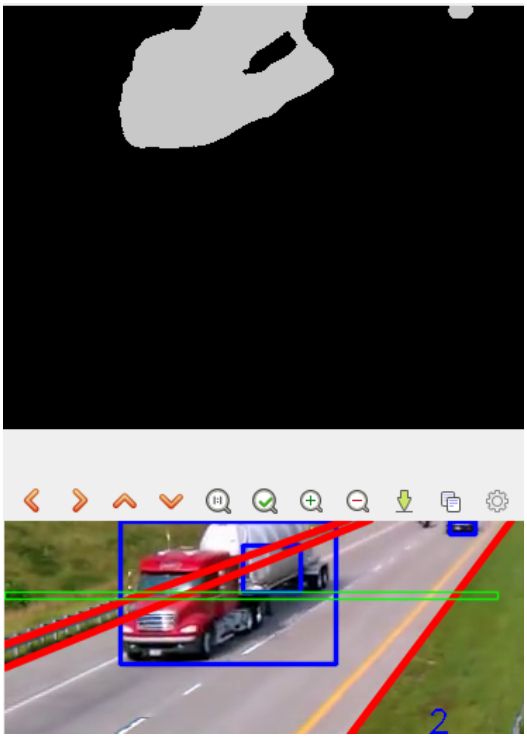
- Transformations morphologiques avec une fermeture et une ouverture (fonctions erode() et dilate()) :
https://docs.opencv.org/4.x/db/df6/tutorial_erosion_dilatation.html

- Trouver les différents contours afin de séparer les différents véhicules avec la fonction findContours()
https://docs.opencv.org/4.x/d3/dc0/group__imgproc__shape.html#gadf1ad6a0b82947fa1fe3c3d497f260e0

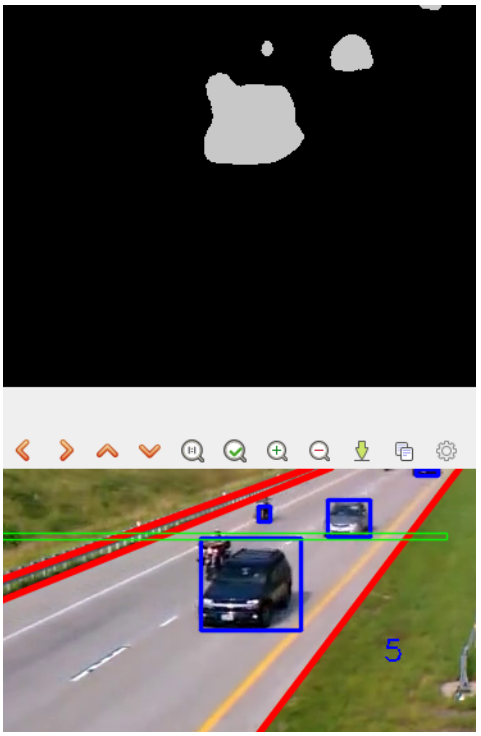
- Pour chaque contour trouvé, calculer un rectangle autour avec boundingRect()
https://docs.opencv.org/4.x/d3/dc0/group__imgproc__shape.html#ga103fcbda2f540f3ef1c042d6a9b35ac7

- Dessiner ce contour avec rectangle() :
https://docs.opencv.org/4.x/d6/d6e/group__imgproc__draw.html

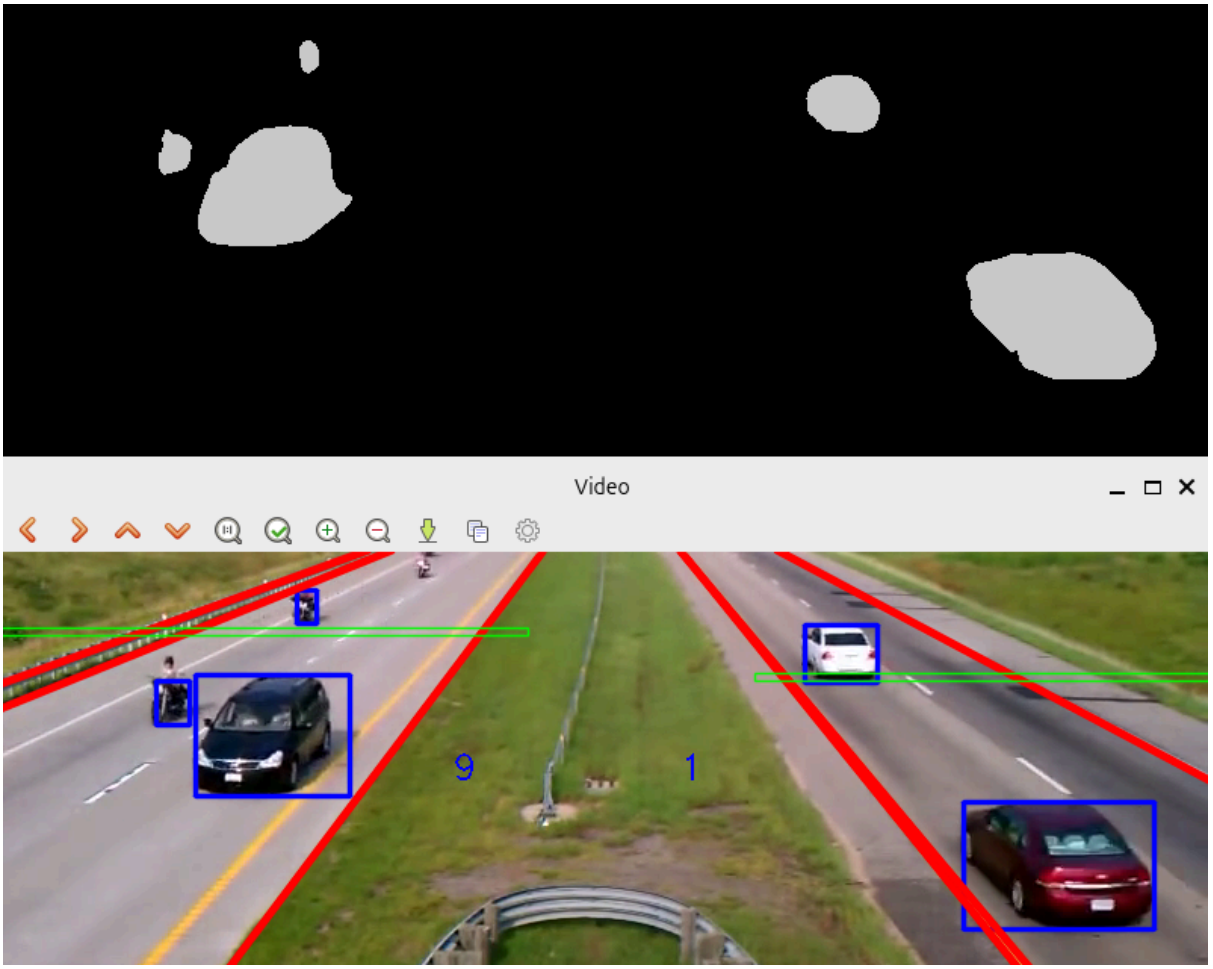
Résultat : Il y a bien un rectangle qui suit les véhicules cependant il y a deux problèmes :
Premièrement, lorsque les véhicules sont trop proches, ils comptent comme un seul véhicule (pas assez d'érosion).
Deuxièmement, pour les camions, l'algorithme détecte parfois plusieurs véhicules au lieu d'un (trop d'érosion).
Ces deux problèmes sont issus des transformations morphologiques (erode et dilate).
J'ai ajusté les paramètres des masques et l'ordre des transformations afin d'obtenir le meilleur compromis.



Cas où il y a trop d’érosion



Cas où il n’y a pas assez d’érosion

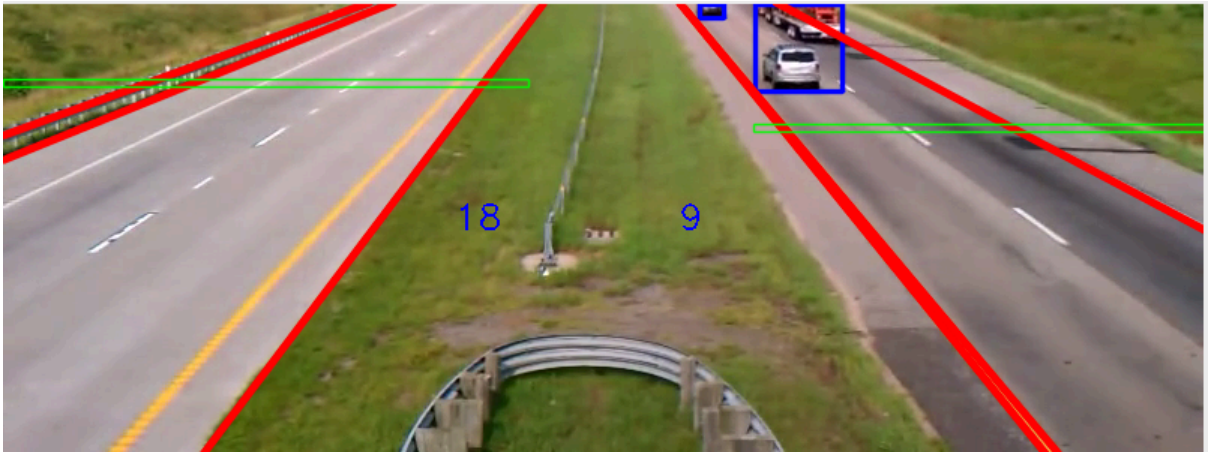


Cas où il n’y a pas de problème.

III. Compter les véhicules :

- Pour chaque rectangle que l'on trace, on vérifie si les coordonnées d'un des angles passe dans une certaine zone prédéfinie.
- Si c'est le cas, on incrémente un compteur.
Il y a deux zones : une pour la voie de gauche et une pour la voie de droite.
Les zones font 4 pixels d'épaisseur : suffisamment grand pour être détecter en cas de bruit mais assez petit pour ne pas être compté plusieurs fois.
- On affiche ensuite les deux compteurs avec putText()
https://docs.opencv.org/4.x/d6/d6e/group__imgproc__draw.html#ga5126f47f883d730f633d74f07456c576

Résultat : Les véhicules sont globalement bien comptés.
Afin d'obtenir de meilleurs résultats, j'ai dû modifier la position des zones de détection de façon à ce que les véhicules soient bien séparés selon l'angle de vue.
J'ai aussi choisi le point de la boundingBox donnant les meilleures performances.



Ainsi le programme compte 18 véhicules pour la voie de gauche et 9 pour la voie de droite.